第34课 阶段三复习与测试

一、课程信息

项目	内容
课次	第34课 / 共40课
主题	阶段三复习 + 阶段测试
目标	系统回顾第25-33课知识点，通过测试检验掌握情况
前置知识	第25-33课（字符串、前缀和与差分、二分查找、递归、汉诺塔、栈与队列、vector与STL、结构体）
后续衔接	第35课 DFS入门（进入阶段四：算法思想）

二、课堂时间分配（共100分钟）

时间段	时长	内容
0′ - 5′	5分钟	开场导入，说明本节课安排
5′ - 15′	10分钟	专题一、二复习：字符串 + 前缀和与差分
15′ - 25′	10分钟	专题三、四复习：二分查找 + 递归与汉诺塔
25′ - 35′	10分钟	专题五、六复习：栈与队列 + vector与STL
35′ - 45′	10分钟	专题七复习：结构体 + 综合串联
45′ - 50′	5分钟	答疑 + 发放测试卷
50′ - 100′	50分钟	**阶段测试**（独立完成）

三、复习提纲

专题一：字符串（第25-26课）

1. char 与 string 的区别

```
char c = 'A';         // 单个字符，用单引号
string s = "Hello";   // 字符串，用双引号
```

老师提醒： 单引号 `'A'` 是字符，双引号 `"A"` 是字符串，千万别搞混！

2. string 的声明与输入输出

```
string s1;            // 声明空字符串
string s2 = "cat";    // 声明并初始化

cin >> s;              // 读入一个单词（遇空格停止）
getline(cin, s);       // 读入一整行（含空格）

cout << s << endl;     // 输出字符串
```

3. 常用操作速查表

操作	写法	说明
求长度	`s.size()` 或 `s.length()`	返回字符个数
访问第i个字符	`s[i]`	下标从0开始
拼接	`s1 + s2`	把两个字符串连起来
比较	`s1 == s2`、`s1 < s2`	按字典序比较
截取子串	`s.substr(起始位置, 长度)`	取一部分出来
查找	`s.find("cat")`	找到返回位置，找不到返回 `string::npos`

4. 遍历字符串

```
// 方法一：下标遍历
for (int i = 0; i < s.size(); i++) {
    cout << s[i];
}

// 方法二：范围for（了解）
for (char c : s) {
    cout << c;
}
```

课堂提问： `string s = "abcde";`，`s[2]` 是什么？`s.size()` 是多少？

> 答：`s[2]` 是 `'c'`，`s.size()` 是 `5`。

专题二：前缀和与差分（第27课）

1. 前缀和：快速求区间和

核心思想： 提前算好"从头到每个位置的总和"，之后求任意区间和只需做一次减法。

```
// 构建前缀和数组（下标从1开始）
int a[N], pre[N];  // a是原数组，pre是前缀和数组
pre[0] = 0;
for (int i = 1; i <= n; i++) {
    pre[i] = pre[i - 1] + a[i];
}

// 求区间 [L, R] 的和
int sum = pre[R] - pre[L - 1];
```

生活比喻： 就像存折上记录的"累计存款"，想知道某两个月之间存了多少钱，用后面的减前面的就行。

2. 差分：快速给区间加值

核心思想： 想让区间 `[L, R]` 都加上 `v`，只需要修改两个位置。

```
// 差分操作
d[L] += v;
d[R + 1] -= v;

// 最后求前缀和还原数组
for (int i = 1; i <= n; i++) {
    d[i] = d[i] + d[i - 1];  // d[i] 就是最终的 a[i]
}
```

记忆口诀： "前缀和查区间，差分改区间"。

专题三：二分查找（第28课）

1. 核心条件

二分查找要求数组**已经排好序**（升序或降序）。

2. 模板代码

```
// 在升序数组 a 中查找 target
int left = 1, right = n;
while (left <= right) {
    int mid = (left + right) / 2;
    if (a[mid] == target) {
        cout << "找到了，位置是 " << mid << endl;
        break;
    } else if (a[mid] < target) {
        left = mid + 1;   // 目标在右半边
    } else {
        right = mid - 1;  // 目标在左半边
    }
}
if (left > right) {
    cout << "没找到" << endl;
}
```

3. 关键理解

要素	说明
时间复杂度	O(log n)，比逐个查找快得多
前提条件	数组必须有序
每次缩小一半	1000个数最多找10次，100万个数最多找20次

生活比喻： 猜数字游戏，每次猜中间值，根据"大了/小了"缩小范围。

专题四：递归（第29-30课）

1. 递归三要素

要素	说明	比喻
函数自己调用自己	在函数体内调用自身	照镜子看到镜中镜
递归出口（基线条件）	什么时候停下来	到底楼了就不再下楼梯
问题规模缩小	每次调用都离出口更近	每次下一级台阶

2. 经典例子：阶乘

```
int factorial(int n) {
    if (n == 0 || n == 1) return 1;  // 出口
    return n * factorial(n - 1);      // 缩小规模
}
```

调用过程（以 `factorial(4)` 为例）：

```
factorial(4) = 4 * factorial(3)
             = 4 * 3 * factorial(2)
             = 4 * 3 * 2 * factorial(1)
             = 4 * 3 * 2 * 1
             = 24
```

3. 汉诺塔

```
void hanoi(int n, char from, char via, char to) {
    if (n == 1) {
        cout << from << " -> " << to << endl;
        return;
    }
    hanoi(n - 1, from, to, via);   // 把上面n-1个从from借to移到via
    cout << from << " -> " << to << endl;  // 把最大的从from移到to
    hanoi(n - 1, via, from, to);   // 把n-1个从via借from移到to
}
```

盘子数 n	移动次数
1	1
2	3
3	7
n	2^n - 1

专题五：栈与队列（第31课）

1. 栈（Stack）—— 后进先出（LIFO）

生活比喻： 叠盘子，最后放上去的最先拿走。

```
#include <stack>

stack<int> st;
st.push(10);     // 压入栈顶
st.push(20);
st.push(30);
cout << st.top();   // 30（看栈顶）
st.pop();           // 弹出30
cout << st.top();   // 20
cout << st.size();  // 2
cout << st.empty(); // 0（不为空）
```

2. 队列（Queue）—— 先进先出（FIFO）

生活比喻： 排队买饭，先排的先买到。

```
#include <queue>

queue<int> q;
q.push(10);       // 入队（排到队尾）
q.push(20);
q.push(30);
cout << q.front(); // 10（看队首）
cout << q.back();  // 30（看队尾）
q.pop();           // 出队（10离开）
cout << q.front(); // 20
```

3. 对比速查

操作	栈 stack	队列 queue
添加元素	`push()`	`push()`
删除元素	`pop()`（删栈顶）	`pop()`（删队首）
查看元素	`top()`（栈顶）	`front()`（队首）/ `back()`（队尾）
大小	`size()`	`size()`
判空	`empty()`	`empty()`

专题六：vector与常用STL（第32课）

1. vector：可变长度的数组

```
#include <vector>

vector<int> v;          // 声明空vector
vector<int> v(5, 0);    // 5个元素，初始值都是0

v.push_back(10);  // 尾部添加
v.pop_back();     // 尾部删除
v.size();         // 元素个数
v[i];             // 访问第i个元素（下标从0开始）
v.clear();        // 清空
```

2. 常用STL算法（需要 `#include <algorithm>`）

```
sort(v.begin(), v.end());            // 升序排序
sort(v.begin(), v.end(), greater<int>()); // 降序排序
reverse(v.begin(), v.end());         // 翻转
*max_element(v.begin(), v.end());    // 最大值
*min_element(v.begin(), v.end());    // 最小值
```

3. vector vs 数组

特性	普通数组 `int a[100]`	`vector<int> v`
大小	固定	可变（自动扩容）
添加元素	不方便	`push_back()`
知道有多少元素	需要额外变量记录	`v.size()`

专题七：结构体（第33课）

1. 定义与使用

```
struct Student {
    string name;
    int age;
    int score;
};

int main() {
    Student a;
    a.name = "小明";
    a.age = 10;
    a.score = 95;

    // 也可以用大括号初始化
    Student b = {"小红", 9, 88};

    cout << a.name << " " << a.score << endl;
    return 0;
}
```

2. 结构体数组

```
Student stu[100];
for (int i = 0; i < n; i++) {
    cin >> stu[i].name >> stu[i].age >> stu[i].score;
}
```

3. 结构体排序（自定义比较函数）

```
bool cmp(Student a, Student b) {
    return a.score > b.score;  // 按成绩从高到低排序
}

sort(stu, stu + n, cmp);
```

老师提醒： 结构体就像一个"自定义的收纳盒"，可以把不同类型的数据装在一起。

四、阶段三知识全景图

```
阶段三：数据结构与进阶
├── 字符串处理
│   ├── char 与 string
│   ├── 输入输出（cin/getline）
│   └── 常用操作（拼接、截取、查找、遍历）
├── 算法技巧
│   ├── 前缀和（快速求区间和）
│   ├── 差分（快速区间修改）
│   └── 二分查找（有序数组中快速查找）
├── 递归思想
│   ├── 递归三要素
│   ├── 阶乘
│   └── 汉诺塔
├── 数据结构
│   ├── 栈（后进先出）
│   ├── 队列（先进先出）
│   └── vector（可变长数组）
├── STL工具箱
│   ├── sort / reverse
│   └── max_element / min_element
└── 结构体
    ├── 定义与使用
    ├── 结构体数组
    └── 自定义排序
```

五、下节课预告

> **第35课：DFS入门**
>
> 我们将正式进入**阶段四：算法思想**！第一站是 DFS（深度优先搜索）。
>
> DFS 就像走迷宫——沿着一条路一直走到底，走不通了就退回来换一条路。
>
> 它会用到我们刚复习的**递归**，所以递归一定要掌握好哦！
>
> 预习小任务：回顾递归三要素，想一想"走迷宫"的过程中，什么是出口？什么是缩小规模？